37

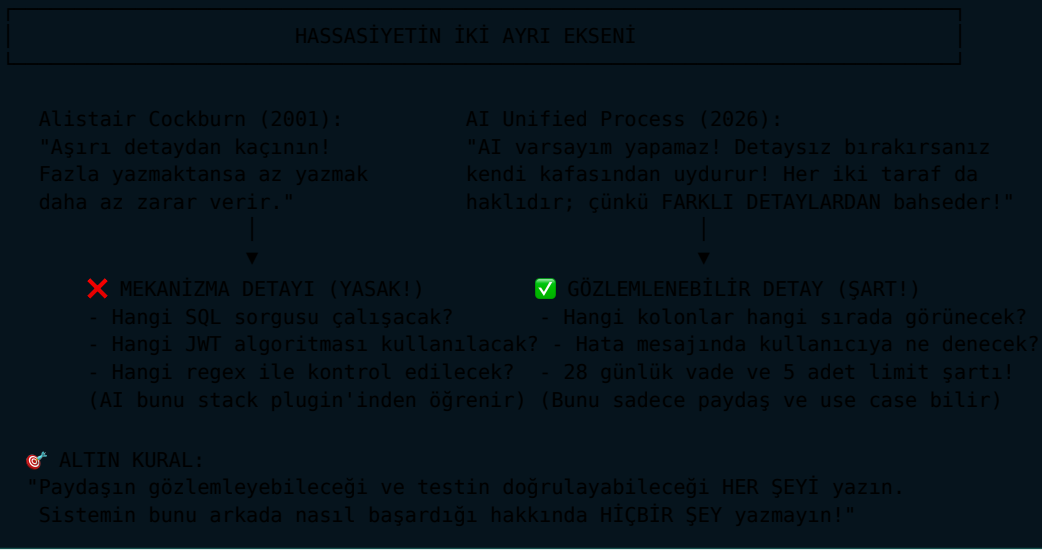
BÖLÜM: YAPAY ZEKÂ İÇİN HASSASIYET: NE SÖYLENMELI, NE DIŞARIDA BIRAKILMALI? | MODULE: PRECISION FOR AI – WHAT TO SAY AND WHAT TO LEAVE OUT

Hassasiyet Paradoksu: Gözlemlenebilir Davranış vs Mekanizma Detayı

The Precision Paradox: Observable Behavior vs Mechanism Detail

⚡ Akış, Varlık & Süreç Şeması

⚡ Process & Artifact Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir restorana gidip şefe sipariş verdiğinizi düşünün: Şefe 'Bana orta pişmiş antrikot, yanında kuşkonmaz ve patates püresi ile saat 20:00'de gelsin' dersiniz (Gözlemlenebilir istek!). Ama şefe 'Fırını 230 dereceye ayarla, tavayı 3 dakika ısıt, tuzu sol elle at' derseniz şef sizi mutfaktan kovar (Mekanizma detayı!). İşte use case şefe ne istediğinizi söyler, mutfak aletlerinin kullanımını şefe bırakır!

🇬🇧 ENGLISH (ELI5)

Imagine ordering at a fine restaurant: You tell the chef 'Bring me medium-rare ribeye steak with grilled asparagus and mashed potatoes at 8:00 PM' (Observable requirement!). But if you instruct the chef: 'Set the stove to 230 degrees, preheat the pan for 3 minutes, and sprinkle salt with your left hand', the chef kicks you out (Mechanism detail!). The use case defines what lands on the table; it lets the kitchen execute the technique.

🔍 MAANG & AIUP Seviyesi Teknik Derinlik

🇹🇷 AIUP & MAANG DERİNLEMESİNE ANALİZ

Hassasiyet Paradoksunun Çözümü: Klasik use case literatüründe Cockburn, detay yazmanın tehlikeleri konusunda onlarca sayfa uyarıda bulunmuştur. Detay belgeyi kırılganlaştırır, bakım maliyetini artırır ve yetenekli geliştiricilerin yaratıcılığını çalar. Ancak günümüz AI destekli dünyasında Martinelli, 'Kitap listesini gösterir' gibi gevşek adımların AI tarafından tamamen rastgele kolonlarla üretilmesine sebep olduğunu gösterir. Çözüm, detayı tek bir eksen değil, iki zıt eksen olarak görmektir: 1) Gözlemlenebilir Detay (Observable Detail): Kullanıcının ekranda gördüğü veri, sıralama ölçütü, iş mantığı sınırları (28 gün, 5 kitap). Bu detay eksik bırakılırsa AI boşluğu kendi tahminleriyle doldurur ve sistem mimarisi çöker. 2) Mekanizma Detayı (Mechanism Detail): HTTP fiilleri, SQL transaction izolasyon seviyeleri, hash algoritmaları, regex desenleri. Bu detay yazılırsa şartname koda bağımlı hale gelir ve yeniden üretilebilirlik (regenerability) yok olur.

🔍 MAANG & AIUP Technical Deep-Dive

🇬🇧 AIUP & MAANG TECHNICAL DEEP-DIVE

Resolving the Precision Paradox: Cockburn warned extensively against over-specification because detail in traditional engineering is expensive to maintain, fragile against refactoring, and strips decisions from experienced developers. However, generative AI inverts this dynamic: an under-specified step like 'System displays books' forces the LLM to invent arbitrary table columns, sorting heuristics, and layout fields. Martinelli resolves this tension via a single axiom: 'Specify everything a stakeholder could observe and a test could verify. Specify nothing about how the system achieves it.' Detail is not one-dimensional; it splits into observable outcomes (required) versus underlying mechanisms (strictly banned).

⚡ Modern Java 25 & Clean Architecture Uygulaması ⚡ Java 25 & Clean Architecture Implementation

```java
// Java 25: Gözlemlenebilir İş Sonucunun Mekanizmadan Ayrılması
package com.maang.aiup.precision;

import java.time.LocalDate;
import java.util.List;

public class PrecisionParadoxDemo {
    // 1. GÖZLEMLENEBİLİR İŞ SONUCU (Paydaş ve Test Bunu Görür!)
    public record BorrowConfirmationView(
        String bookTitle,
        String memberName,
        LocalDate dueDate,
        int remainingLoanQuota
    ) {}

    // 2. MEKANİZMA (Use case'de ASLA geçmez; Spring Data, SQL, Redis buraya aittir)
    public interface LoanRepositoryPort {
        void persistLoanRecord(String memberId, String bookId, LocalDate dueDate);
    }

    // 3. USE CASE İCRA EDİCİSİ (Sadece gözlemlenebilir kuralı işletir)
    public static class BorrowBookInteractor {
        public BorrowConfirmationView execute(String memberName, String title, int currentLoans) {
            if (currentLoans >= 5) { // BR-005 kuralı
                throw new IllegalStateException("Ödünç limiti aşıldı! En fazla 5 kitap.");
            }
            LocalDate dueDate = LocalDate.now().plusDays(28); // 28 gün kuralı
            return new BorrowConfirmationView(title, memberName, dueDate, 5 - (currentLoans + 1));
        }
    }

    void main() {
        var interactor = new BorrowBookInteractor();
        var result = interactor.execute("Alice", "Designing Data-Intensive Apps", 2);
        System.out.println("✅ Gözlemlenebilir Sonuç: " + result);
    }
}
```

⚠️ AI Yanılsaması, Mekanizma Sızıntısı & Mülakat Tuzağı ⚠️ AI Hallucination, Mechanism Leak & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE AI HALÜSİNASYONU

'Sistem kullanıcıyı uygun şekilde bilgilendirir' veya 'verileri gösterir' gibi yuvarlak ifadeler kullanmayın. Bu kelimeler AI'a kafasına göre uydurma yetkisi (blank check) verir!

🇬🇧 INTERVIEW TRAP & AI HALLUCINATION

Never use vague escape phrases like 'System notifies user appropriately' or 'System shows relevant data'. Every ambiguous word delegates architecture decisions to AI halluncinations.

🎯 Mülakatta ve Üretimde Hatırlanacak Altın Kurallar

• Paydaşın görebileceği ve testin doğrulayabileceği her şeyi somut olarak yazın.

• Sistemin bunu nasıl yaptığı (SQL, JWT, regex) use case şartnamesine asla yazılamaz.

• Eksik gözlemlenebilir detay, AI'ın kafasına göre kod uydurmasına davetiye çıkarır.

🎯 Key Architectural Takeaways

• Specify everything a stakeholder can observe and an acceptance test can verify.

• Never specify implementation mechanisms (SQL, JWT, regex) in use case specifications.

• Omitted observable details force the AI to invent business behavior arbitrarily.